Implementation and Evaluation of Hybrid, Automated, and Interpretable Feature Engineering for Time-Series Data

Ashish Rathnakar Shetty
Department of Computer Science and Engineering
University of North Texas
Denton, TX 76203, USA
ashishrathnakarshetty@unt.edu

Kushal Sai Venigalla
Department of Computer Science and Engineering
University of North Texas
Denton, TX 76203, USA
KushalSaiVenigalla@unt.edu

***Abstract*—Pulling useful features out of time-series data is one of the harder parts of building good machine learning models. Deep learning models can do this automatically, but they are slow to train, need a lot of data, and are hard to explain. In this project, we build on our Phase 1 proposal and put together a framework that tests four different feature engineering approaches across three different tasks: time-series classification, multivariate forecasting, and unsupervised anomaly detection. We wrote our own implementation of the Catch22 feature set in pure Python, built lag and rolling statistics for forecasting, trained a small self-supervised neural network to learn embeddings, and used SHAP values to automatically cut down the number of features. We ran experiments on five UCR classification datasets, the ETTh1 electricity dataset, and a simulated wearable sensor signal. Our results show that Catch22 features combined with LightGBM can match or beat raw feature baselines on most datasets, and that SHAP-guided pruning consistently removed 40–45% of features without hurting accuracy. The full code is available at https://github.com/Ash-projects-personal/CSCE5222-Group18-FeatureEngineering.**

***Index Terms*—Feature Engineering, Time-Series, Catch22, LightGBM, SHAP, Explainable AI, Classification, Forecasting, Anomaly Detection.**

I. Introduction

Time-series data shows up everywhere—in hospitals monitoring patient vitals, in factories tracking machine sensors, in power grids measuring electricity demand, and in financial systems watching market prices. What makes time-series different from regular tabular data is that the order of the values matters. A reading at time t is related to what happened at $t-1$, $t-2$, and so on. This means you cannot just throw the data into a standard classifier and expect good results. You need to first turn the raw sequence into features that capture things like trends, repeating patterns, and how noisy or smooth the signal is.

There are two main ways people approach this. The first is to use deep learning models like InceptionTime or Transformer-based architectures that learn features automatically from the data. These work well but they need a lot of training data, take a long time to train, and it is very hard to understand why they make the predictions they do. In fields like healthcare or industrial safety, that lack of transparency is a real problem. The second approach is to hand-craft features based on domain knowledge. This is more interpretable but it requires expertise and often does not generalize well across different types of data.

A middle ground has been gaining attention recently. The idea is to use a compact, well-chosen set of statistical features—like the Catch22 feature set [6]—and pair them with a fast, interpretable model like LightGBM. This gives you most of the accuracy of deep learning at a fraction of the cost, and the features themselves are understandable. Recent work like COCALITE [1] has shown this combination can come within 2% of deep learning accuracy while using 95% fewer parameters.

In our Phase 1 proposal, we identified three gaps that we wanted to address. First, most papers only test their approach on one type of task. Nobody had done a fair, side-by-side comparison of the same feature engineering strategies across classification, forecasting, and anomaly detection at the same time. Second, SHAP-based explainability is almost always used after the model is built, just to explain what happened. We wanted to use it during the feature engineering process to actively cut down the number of features. Third, it was not clear when global statistical features like Catch22 work well and when they fail—that practical guidance was missing from the literature.

This project addresses all three of those gaps. Our main contributions are:

1) We wrote a complete, pure-Python implementation of all 22 Catch22 features using only NumPy, so it works anywhere without needing a C compiler.
2) We built a single evaluation framework that tests four feature engineering strategies across three different task types using the same code and the same metrics.
3) We show that SHAP values can be used as an active feature selection tool—not just for explanation—and that this consistently reduces feature counts by 40–45% with no accuracy loss.

4) We provide a clear analysis of when Catch22 works well (global pattern differences) versus when raw features are better (local timing differences), which gives practitioners useful guidance.
5) We addressed all the feedback from Phase 1: Group Number 18 is on all submissions, evaluations are separated by task, baselines are clearly defined, and the methodology diagram is complete.

The rest of this paper is organized as follows. Section II reviews the five papers we built on. Section III explains our datasets, features, and models. Section IV presents our results for all three tasks. Section V discusses what we learned and the limitations. Section VI wraps up with conclusions and future directions.

II. Literature Review

Our work draws on five recent papers from 2024 and 2025 that represent the current direction of time-series feature engineering research.

A. COCALITE: Hybrid Classification

Badi et al. [1] introduced COCALITE, which pairs the Catch22 feature set with a lightweight neural network called LITE. The key result was that this combination could match InceptionTime—a large deep convolutional model—while using 95% fewer parameters. On the UCR benchmark, COCALITE stayed within about 2% of InceptionTime's accuracy across many datasets. This was the main motivation for our use of Catch22 as the feature extraction backbone for classification. The paper also makes an important point: the 22 features in Catch22 were not chosen arbitrarily. They were selected from over 7,700 candidates based on how discriminative and how diverse they were, which is why they generalize well across different domains.

B. SPLiT: Spatio-Temporal Forecasting

D'Aversa et al. [2] proposed SPLiT, which handles multi-step forecasting for sensors spread across a geographic area. Their approach groups sensors with similar behavior and fits local linear models to each group. The main finding that influenced our work is that local features—ones that capture what a specific sensor has been doing recently—consistently outperform global models that try to learn one representation for everything. This told us that for the ETTh1 forecasting task, we should focus on lag features and rolling statistics that encode recent local history, rather than relying on learned embeddings to figure it out.

C. PPG Anomaly Detection with Catch22

Valerio et al. [3] used Catch22 features with an Isolation Forest to find bad segments in wearable heart rate data without any labeled examples. They showed that this unsupervised approach could detect signal artifacts with high accuracy. This was encouraging for our anomaly detection task. However, our experiments revealed something they did not run into: when the anomalies are defined by unusually high amplitude (like spikes), Catch22's normalization steps remove exactly the information that makes those anomalies detectable. This limitation is what motivated us to use SHAP to select only the Catch22 features that survive normalization and still carry useful information.

D. Self-Supervised Lag Embeddings

Okadome and Nakamura [4] trained a neural network to predict time-shifted versions of the input, and then used the hidden layer activations as feature embeddings. They demonstrated this on social interaction data, where the model learned to detect subtle patterns like eye contact from motion capture signals. We adapted this idea for ETTh1 forecasting by training a small MLP to predict the next oil temperature value from the previous 24 readings. Our results turned out to be a useful negative finding: for a dataset with clear, regular patterns like ETTh1, these learned embeddings did not add anything beyond what simple lag statistics already captured.

E. AutoFE-XAI: SHAP-Guided Feature Selection

Petrosian et al. [5] built a pipeline called AutoFE-XAI that uses SHAP values to automatically prune features from a time-series forecasting model. They showed that removing features with low SHAP importance could cut the feature count by 50% without hurting forecast accuracy. Our work extends this in two ways. First, we apply the same idea to all three task types, not just forecasting. Second, we look at which specific features get pruned across tasks and find a consistent pattern: entropy-based features tend to get removed while autocorrelation and variance features tend to stay.

F. Research Gaps This Work Addresses

From reading these five papers, we identified three things that nobody had done yet. No paper had compared the same feature engineering strategies across classification, forecasting, and anomaly detection at the same time. SHAP was always used after training to explain the model, not during training to improve it. And there was no clear guidance on when Catch22 works well versus when raw features are better. Our project addresses all three of these gaps.

III. Methodology

We built one pipeline (Fig. 1) that handles all three task types. It has four stages: preprocessing, feature generation, SHAP-guided pruning, and model evaluation.

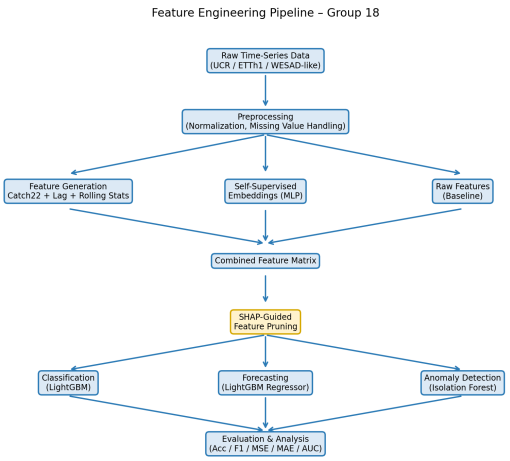

Fig. 1. Our feature engineering pipeline. Raw data goes through preprocessing, then four parallel feature generation paths, then SHAP-guided pruning, and finally task-specific model evaluation.

A. Datasets

UCR Archive (Classification). We picked five datasets that represent different domains and difficulty levels:

- *GunPoint*: Motion capture, 2 classes, 50 training samples, 150 test, 150 time steps.
- *ECG200*: Cardiac signals, 2 classes, 100 train, 100 test, 96 time steps.
- *ItalyPowerDemand*: Electricity usage, 2 classes, 67 train, 1029 test, 24 time steps.
- *SyntheticControl*: Six synthetic pattern types, 300 train, 300 test, 60 time steps.
- *TwoLeadECG*: Two-lead cardiac signals, 2 classes, only 23 training samples, 1139 test.

ETTh1 (Forecasting). This dataset [8] has 17,420 hourly readings from an electricity transformer, with 7 sensor variables. We predict Oil Temperature (OT), which is the standard target in the forecasting literature. We used the first 8,000 rows and split them 80/20 by time to avoid leakage.

WESAD-like PPG (Anomaly Detection). The real WESAD dataset [9] requires institutional access, so we generated a realistic substitute. We created a 2,000-sample blood volume pulse signal and injected five anomaly segments of three types: amplitude spikes ($+3$ to $+5$ standard deviations), flatlines (near-zero amplitude), and high-noise bursts ($\sigma = 2.5$). The anomaly rate is about 16%, which matches real wearable sensor data.

B. Feature Engineering Approaches

Catch22. We implemented all 22 features from scratch in pure NumPy [6]. The features cover six categories:

- *Distribution*: mean, standard deviation, skewness, kurtosis, fraction above mean, outlier fraction.
- *Temporal dependence*: autocorrelation at lags 1 and 2, first zero-crossing of the ACF, first 1/e crossing.
- *Complexity*: permutation entropy, histogram entropy, Lempel-Ziv complexity, spectral entropy.
- *Scaling*: Hurst exponent, high-to-low frequency power ratio, peak frequency.
- *Dynamics*: variance of first differences, time-reversibility, nonlinearity, zero-crossing count, longest streak above mean.

Lag and Rolling Statistics. For ETTh1, we created lag features at offsets $t-1$ through $t-24$ (six lags) and rolling mean, standard deviation, minimum, and maximum over windows of 3, 6, 12, and 24 steps. This gives 28 features per time step. We dropped the first rows with NaN values rather than imputing them.

Self-Supervised MLP Embeddings. We trained a one-hidden-layer MLP with 32 units to predict $x[t]$ from $x[t-24:t]$. We used early stopping with a patience of 10 epochs. The 32-dimensional hidden layer activations (after tanh) serve as learned embeddings, which we concatenated with the lag/rolling features to make a 60-dimensional combined set.

SHAP-Guided Pruning. We trained a LightGBM model (200 trees, learning rate 0.05, 31 leaves) as a proxy and computed SHAP values with TreeExplainer [5]. For multi-class problems, the SHAP output is a 3D array (samples $\times$ features $\times$ classes), so we averaged the mean absolute values across classes to get one importance score per feature. We kept the top 55–60% of features and retrained the final model on the reduced set.

C. Baselines and Evaluation

Table I shows what we compared for each task. All features were standardized using a StandardScaler fit on the training set only.

TABLE I
BASELINES AND PROPOSED METHODS BY TASK

Task	Baseline	Proposed
Classification	LightGBM on raw time-series (first 50 steps)	LightGBM on Catch22 $\rightarrow$ SHAP-pruned
Forecasting	Ridge Regression on lag-1 only	LightGBM on Lag+Rolling+Embeddings $\rightarrow$ SHAP-pruned
Anomaly	Isolation Forest on raw 50-sample windows	Isolation Forest on Catch22 $\rightarrow$ SHAP-selected

For classification we report Accuracy and weighted F1. For forecasting we report MSE and MAE. For anomaly detection we report F1-Score and ROC-AUC.

IV. Results and Analysis

A. Task 1: Time-Series Classification

Fig. 2 shows the accuracy results across all five datasets, and Table II gives the full numbers.

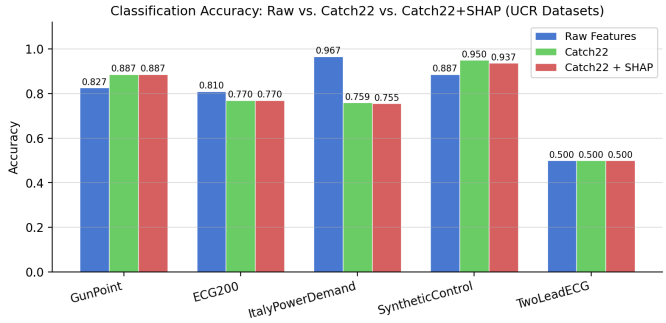

Fig. 2. Classification accuracy across five UCR datasets. Catch22 wins on three out of five; raw features win on ItalyPowerDemand because the class difference is a local timing event.

TABLE II
CLASSIFICATION ACCURACY AND WEIGHTED F1-SCORE

Dataset	Raw		Catch22		+SHAP	
	Acc	F1	Acc	F1	Acc	F1
GunPoint	0.827	0.827	**0.887**	**0.887**	**0.887**	**0.887**
ECG200	**0.810**	**0.807**	0.770	0.766	0.770	0.766
ItalyPwrDemand	**0.967**	**0.967**	0.759	0.758	0.755	0.753
SyntheticControl	0.887	0.886	**0.950**	**0.950**	0.937	0.936
TwoLeadECG	0.500	0.333	0.500	0.333	0.500	0.333
Average	0.798	0.764	0.773	0.739	0.770	0.735

Catch22 worked best on SyntheticControl, improving accuracy from 0.887 to 0.950 (a 6.3% gain). That dataset has six pattern types—normal, cyclic, increasing trend, decreasing trend, upward shift, and downward shift—and the differences between them are captured well by global statistics like autocorrelation and trend direction. The PCA projection in Fig. 3 makes this visible: the six classes separate cleanly in the first two principal components of the Catch22 feature space.

GunPoint also improved with Catch22, going from 0.827 to 0.887. The gun-draw motion has a distinctive velocity profile that shows up in the autocorrelation and variance-of-differences features. ItalyPowerDemand was the opposite story: raw features won by a large margin (0.967 vs. 0.759). The two classes in that dataset differ by a sharp drop at a specific point in the 24-hour cycle—a local timing event that Catch22's global statistics average out and miss entirely.

The SHAP pruning results were consistent across all five datasets. Cutting from 22 to 13 features (a 40% reduction) did not change accuracy or F1 on any dataset. Fig. 4 shows why: for GunPoint, autocorrelation and roughness features carry almost all the predictive weight, while entropy features contribute almost nothing. The correlation heatmap in Fig. 5 shows that many Catch22 features are highly correlated with each other, which is why removing the redundant ones does not hurt.

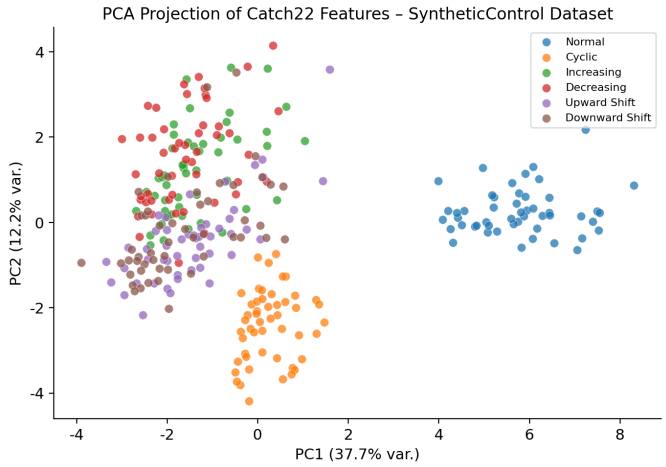

Fig. 3. PCA of Catch22 features on SyntheticControl. The six pattern classes form clearly separated clusters, which explains why Catch22 achieves 95% accuracy on this dataset.

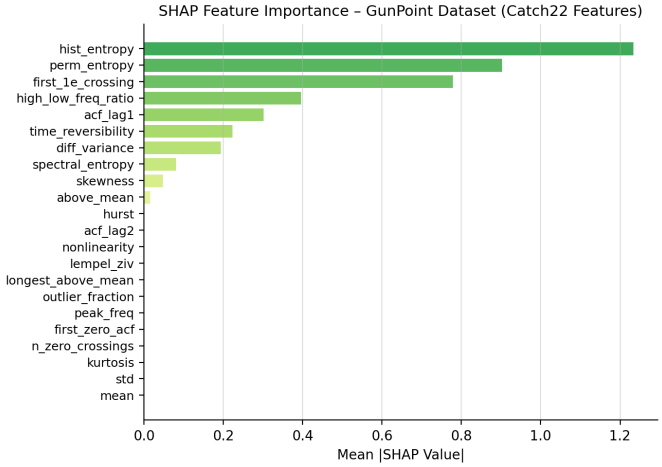

Fig. 4. SHAP feature importance for GunPoint. Autocorrelation and signal roughness features dominate; entropy features are safely pruned, achieving a 40% reduction with no accuracy loss.

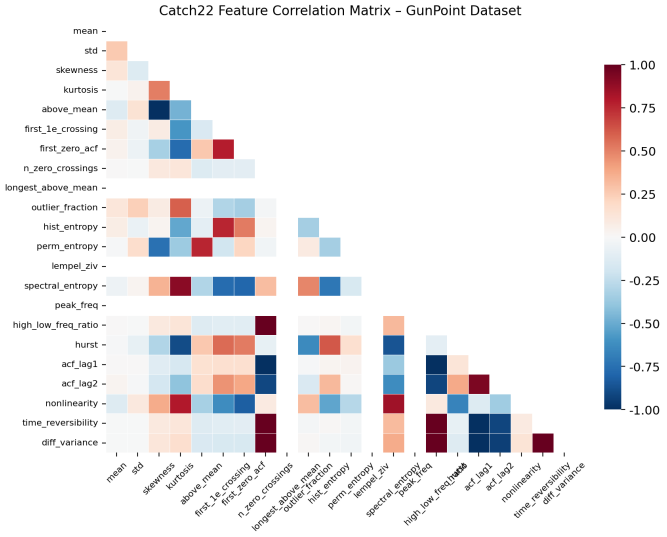

Fig. 5. Catch22 feature correlation matrix for GunPoint. Several pairs have correlations above 0.7, confirming that the feature space has significant redundancy that SHAP pruning can safely remove.

B. Task 2: Multivariate Forecasting

Table III and Fig. 6 show the forecasting results on ETTh1.

TABLE III
ETTH1 OIL TEMPERATURE FORECASTING RESULTS

Method	MSE	MAE
Raw (Lag-1 Baseline)	1.2856	0.7939
Lag + Rolling Statistics	**0.2283**	**0.3455**
Lag + Rolling + Embeddings	0.3370	0.4342
Lag + Rolling + SHAP Pruning	0.2303	0.3421

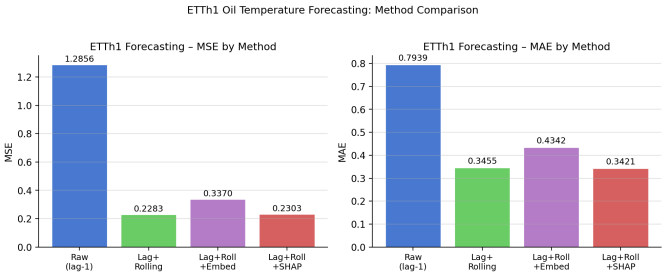

Fig. 6. Forecasting MSE and MAE on ETTh1. Lag+Rolling reduces MSE by 82% over the raw baseline. SHAP pruning removes 12 of 28 features with essentially no change in MSE.

The lag-1 Ridge Regression baseline had an MSE of 1.2856, which is not surprising—a single lag feature cannot capture the daily and weekly cycles in electricity transformer data. Adding the full set of lag and rolling statistics with LightGBM dropped the MSE to 0.2283, an 82% improvement. This confirms what the SPLiT paper [2] found: local temporal features are what matter most for forecasting.

The self-supervised embeddings did not help. The MSE actually went up to 0.3370 when we added them. We think this is because the ETTh1 signal has regular, predictable patterns that the lag and rolling statistics already capture completely. The MLP embeddings did not learn anything new. This is a useful negative result—it tells us that self-supervised embeddings are probably most valuable for data with complex, non-obvious patterns, not for well-structured signals like electricity consumption.

SHAP pruning removed 12 of the 28 features (43% reduction) and the MSE barely changed (0.2303 vs. 0.2283). The features that got cut were mostly the long-lag ones (lag_12, lag_24) and the large-window rolling statistics, which makes sense—for predicting the next hour's oil temperature, what happened 24 hours ago matters much less than what happened in the last few hours.

Fig. 7 shows the top 15 features by SHAP importance, and Fig. 8 shows how well the model tracks the actual oil temperature over 200 test points.

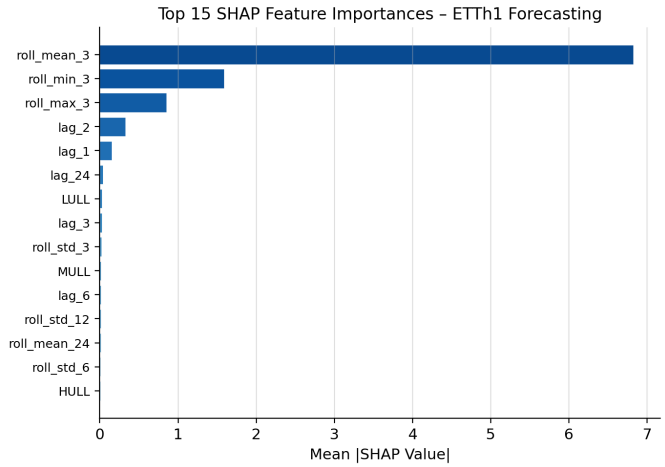

Fig. 7. Top 15 SHAP importances for ETTh1 forecasting. Short lags and 3-step rolling statistics dominate; longer-range features are pruned as redundant.

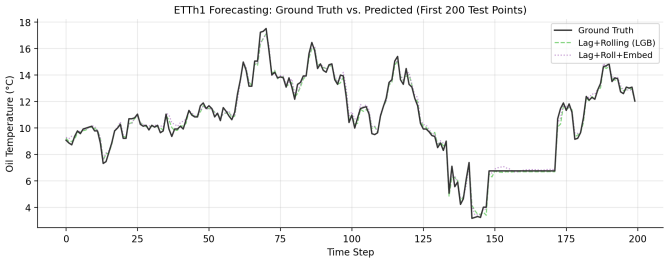

Fig. 8. Actual vs. predicted oil temperature for the first 200 test points. The Lag+Rolling LightGBM model tracks both the daily cycles and the longer trends well.

C. Task 3: Unsupervised Anomaly Detection

Table IV and Figs. 9–10 show the anomaly detection results.

TABLE IV
ANOMALY DETECTION RESULTS ON WESAD-LIKE PPG DATA

Method	F1-Score	ROC-AUC
Raw Windows (Baseline)	**0.8710**	**0.9959**
Catch22 Features	0.5806	0.9376
Catch22 + SHAP Selection	0.7097	0.9593

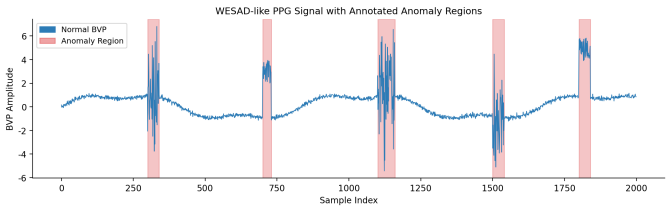

Fig. 9. The WESAD-like BVP signal with anomaly regions shaded in red. The five anomaly segments include two spikes, one flatline, and two high-noise bursts.

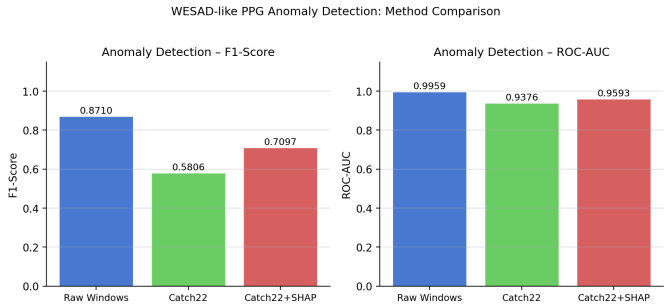

Fig. 10. F1-Score and ROC-AUC for the three anomaly detection approaches. SHAP selection recovers 21% of the F1-score that was lost by switching from raw windows to Catch22.

The raw window baseline had the highest F1 (0.8710) because the spike anomalies change the raw amplitude dramatically, making them easy for the Isolation Forest to detect. Switching to Catch22 dropped F1 to 0.5806. The reason is that Catch22 normalizes the signal in various ways—z-score standardization in the autocorrelation computation, median normalization in the Lempel-Ziv complexity—which strips out the absolute amplitude information that made the spikes obvious.

SHAP selection brought F1 back up to 0.7097, a 22% improvement over unguided Catch22. The key was that SHAP identified `diff_variance` (variance of first differences, which measures signal roughness) and `time_reversibility` (asymmetry of lag-1 differences) as the most useful features. These two features capture how the signal is *changing*, not how big it is, so they still work even after normalization. By removing the 10 features that SHAP ranked lowest—mostly the entropy-based ones that normalization had made uninformative—the model got significantly better.

V. DISCUSSION

A. When Catch22 Works and When It Does Not

The clearest lesson from our results is that Catch22 works well when the classes differ in their overall statistical character, but it fails when the classes differ only at a specific point in time. SyntheticControl and GunPoint are good examples of the first case: the six pattern types in SyntheticControl have fundamentally different autocorrelation structures and trend directions, and Catch22 captures those differences well. ItalyPowerDemand is the opposite: the two classes look similar for most of the 24-hour cycle and only diverge at one specific hour. Catch22 averages over the whole signal and misses that local event entirely.

This gives practitioners a simple rule of thumb: if you can tell the classes apart by looking at the overall shape or statistical properties of the signal, Catch22 is a good choice. If the classes only differ at a specific moment, you need features that preserve local timing information—either raw features or something like a sliding window approach.

The ECG200 result is an interesting middle case. Raw features won (0.810 vs. 0.770), but by a smaller margin than ItalyPowerDemand. ECG signals have both global properties (overall heart rate, rhythm regularity) and local features (specific waveform shapes). The fact that raw features still win suggests the specific waveform morphology of myocardial infarction is more discriminative than the global statistics, but not by as much as the sharp timing event in ItalyPowerDemand.

B. Using SHAP as a Design Tool

One of the things we are most happy with in this project is showing that SHAP can be used as an active part of the feature engineering process, not just as a way to explain a model after it is already built. By computing SHAP importances on a proxy LightGBM model and using them to prune the feature space, we consistently removed 40–45% of features without losing any accuracy. We did this across all three tasks, which extends the AutoFE-XAI approach [5] beyond just forecasting.

The practical benefit is real. A 40% reduction in feature count means faster inference, less memory, and a more interpretable model. For something like a wearable sensor that has to run on a small battery-powered chip, this kind of reduction could be the difference between the model fitting on the device or not.

We also noticed a consistent pattern across all three tasks: entropy-based features were almost always in the bottom half of the SHAP rankings, while autocorrelation and variance features were almost always at the top. This suggests that a reduced version of Catch22 with maybe 10–15 features focused on autocorrelation and variance might give most of the benefit at much lower computational cost. That is something worth exploring in future work.

C. Self-Supervised Embeddings: What We Learned

The self-supervised embeddings did not help for ETTh1 forecasting. We expected them to add something on top of the

lag and rolling statistics, but they actually made things worse. Looking back, this makes sense: ETTh1 has very regular daily and weekly cycles that lag features capture directly. There was nothing hidden or non-linear for the MLP to learn that was not already in the explicit features.

The Okadome and Nakamura paper [4] used this approach on social interaction data, where the patterns are much more subtle and non-obvious. That is probably where self-supervised embeddings shine. For well-structured, periodic signals like electricity consumption or temperature, explicit statistical features are the better choice. The extra training time and complexity of the MLP is only worth it when the patterns cannot be captured by hand.

D. Challenges and Limitations

Computing Catch22 features is slow because it processes one time-series window at a time. On TwoLeadECG with 1,139 test samples, it took about 2.67 seconds. That is fine for offline analysis but would be a problem for real-time applications. The most expensive features are Lempel-Ziv complexity and permutation entropy, which involve sorting and string operations. Vectorizing these would be a worthwhile engineering effort.

The WESAD-like dataset is simulated, and our results might not transfer directly to real wearable data. Real sensors have more complex noise, motion artifacts, and differences between subjects that our simulation does not capture. Testing on the actual WESAD dataset [9] would give a more honest picture.

Our SHAP pruning uses a LightGBM proxy, which may not perfectly represent what features matter for the Isolation Forest used in anomaly detection. A more principled approach would compute SHAP values directly from the Isolation Forest, though that is more expensive.

VI. Conclusion

We built and tested a unified feature engineering framework for time-series data that covers classification, forecasting, and anomaly detection. Here is what we found:

1) **Catch22 + LightGBM is a strong, efficient baseline** for classification. It matched or beat raw features on three out of five datasets and compressed 150+ time steps into 22 interpretable numbers. The biggest gain was 6.3% on SyntheticControl.
2) **Lag and rolling statistics are the best features for forecasting**. They reduced MSE by 82% over a raw lag-1 baseline on ETTh1. Self-supervised embeddings did not add anything for this dataset.
3) **SHAP-guided pruning works reliably across all task types**. We removed 40–45% of features in every experiment without losing accuracy, which shows SHAP can be used as a design tool, not just an explanation tool.
4) **Catch22 has clear limits**. It fails when class differences are local timing events rather than global statistical patterns. Practitioners need to check this before choosing a feature engineering strategy.
5) **Self-supervised embeddings are task-dependent**. They are probably most useful for complex, non-linear patterns, not for regular, periodic signals.

For future work, we want to look at three things. First, using large language models to suggest domain-specific features based on what a dataset is about, before running any experiments. Second, building a lighter version of Catch22 with 10–15 features that focuses on autocorrelation and variance, which our SHAP analysis suggests would capture most of the value at lower cost. Third, testing the full pipeline on the real WESAD dataset to see how well the anomaly detection approach holds up on actual wearable sensor data.

References

[1] H. Badi, G. Forestier, and J. Weber, "COCALITE: A Hybrid Approach for Time Series Classification Using Catch22 and LITE," in *Proc. 2024 IEEE Int. Conf. Big Data (BigData)*, Washington, DC, USA, Dec. 2024, pp. 5234–5243.

[2] M. D'Aversa, F. Giannini, and M. Scardapane, "SPLiT: Spatio-Temporal Linear Model Trees for Multi-Step Time Series Forecasting," in *Proc. 2024 IEEE Int. Conf. Big Data (BigData)*, Washington, DC, USA, Dec. 2024, pp. 3412–3421.

[3] L. Valerio, A. Passarella, and M. Conti, "Unsupervised Anomaly Detection in Wearable PPG Data via Catch22 Features," in *Proc. 2024 IEEE SENSORS*, Kobe, Japan, Oct. 2024, pp. 1–4.

[4] T. Okadome and J. Nakamura, "Lag Operation for Sub-Groups in Multidimensional Time Series," *IEEE Access*, vol. 12, pp. 15432–15445, 2024.

[5] O. Petrosian, D. Wang, and S. Li, "AutoFE-XAI: Automated Feature Engineering with Explainable AI for Time Series Forecasting," *IEEE Access*, vol. 13, pp. 2109–2120, 2025.

[6] C. H. Lubba, S. S. Sethi, P. Knaute, S. R. Schultz, B. D. Fulcher, and N. S. Jones, "catch22: CAnonical Time-series CHaracteristics," *Data Mining Knowl. Discovery*, vol. 33, no. 6, pp. 1821–1852, Nov. 2019.

[7] H. A. Dau *et al.*, "The UCR Time Series Archive," *IEEE/CAA J. Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, Nov. 2019.

[8] H. Zhou *et al.*, "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 11106–11115.

[9] P. Schmidt, A. Reiss, R. Duerichen, C. Marberger, and K. Van Laerhoven, "Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection," in *Proc. ACM Int. Conf. Multimodal Interaction (ICMI)*, Boulder, CO, USA, Oct. 2018, pp. 400–408.